1. Retriever. For ***"larchfield audit"***, the one real email (em-12167) ranks first. Everything after it is an unrelated audit, like em-06790 ("2000 meter data audit") and em-04756 ("Operational Analysis Audit Notification"). "Larchfield" appears in only one email in the whole archive, so every other email can only score on "audit". The ranker still has to return five results, so it picks the emails that say "audit" most often relative to their length. em-06790 is 67 words long and says "audit" five times. These results score about 39% of the top hit, but they're returned all the same. For ***"california power crisis"***, the top three results are the same Chronicle article forwarded three times (em-13031, em-13129 and em-12910). Each copy is its own email with the same words in the same counts, so each gets the same score. The ranker doesn't know they're copies, so the copies take three of the five slots.

2. Protocol. The model writes one JSON object inside `<tool_call>` tags, and I parse it with `json.raw_decode` instead of hunting for the closing tag. That way a tag nested inside a string, a truncated call, and a second call are either linted, error, or rejected. Initially, I tried to parse all the content within the opening and closing tags before realizing that searching for the closing tag actually doesn't matter and introduces an additional source of potential error if the closing tag is included in the tool call body, or if the closing tag was written != 1 time.

3. Termination. A reply without a call previously ended the turn. If the reply is empty, I resend the same messages, up to four times. If it announces a call ("I'll…", "Let me…") but doesn't make one, I nudge it with the call format (i.e., "your last reply said what you would do next but contained no <tool_call> block… write that block now or if you already have what you need write the final answer"). This fixed the most common P15 failure.

4. Search. A doc search snippet is the section heading plus the text around the rarest query term (term with the highest IDF). Each snippet is the section header (80 characters) plus up to 440 characters. Each document can contribute at most two results to prevent one document mentioning a term many times from monopolizing the search results. A `read_doc` window is up to 600 tokens or 40 non-blank lines, whichever is hit first. I kept the read windows capped at 600 tokens instead of the 1,500 hard-cap because the live tests cap the entire conversation at 6,000 tokens and I wanted space for more than 4 read calls.

5. Terminal. Each command's output gets to return about an eighth of the context budget to the model. Anything longer pulls the first 60% and last 40% of this budget from the full output. The full output is saved to a file along with ready-made `grep` and `sed` commands, so the model can still search for content that may have been cropped. I chose this over two alternatives: cutting the middle out for good, or summarizing, as these approaches can drop specific lines that may be needed, while still providing a shortened output that can be used to infer high-level information from the terminal command like whether it was successful, and what the rough shape of the response was.

6. Compaction. When the next call won't fit, the harness shrinks the existing context to 60% of the context budget. The harness first replaces old tool call results with a header (the tool used and its status), its size, its first 160 characters, and the path to any .output/ files. If that isn't

enough, the oldest turns are folded into a summary that preserves every constraint, number, name, and ID the user stated word-for-word, as well as progress so far. The system prompt, the request being worked on, and the most recent tool call result are never dropped.

7. Memory. After the final run, 17 of the 20 memory stores were empty, which is right since ordinary questions shouldn't save anything. The harness saved that the user is on the platform team and that the spec means the GridLens API spec (`memory-the-spec persona`), and the employee numbers 30918 and 31908, with the latter being the corrected number (`memory-corrected-number`). These are good examples of what we want to store as memories because they are personal details about the user that will matter in later conversation. The one thing I'd have wanted to save that wasn't is from `doc-office-days` persona: the user said they're a new hybrid employee in the Oakland office. Since the user never asked it to remember anything, saving was left entirely to the model's judgment, and here that judgment failed.

8. Approval. In confirm mode, only tools that can change things ask for approval, and in practice that's just the terminal. I accept that read-only tools are always allowed because their main risk is prompt injection, and approving a search query wouldn't stop an injection anyway; if every read-only tool call required approval then the user would learn to just click yes without reading.

9. Sandbox. On my Mac, `sandbox-exec` blocks programs that aren't on the allowlist, hides API keys, kills commands at the timeout, and stops network access, writes outside the workspace, and reads of my home directory. The harness can run any code through `python3`, and use CPU and disk freely within its timeout. I accept these because the coding tasks need python3. Plus, with no network and no writes outside a temporary workspace, nothing a jailed process reads can leave the machine through the jail.

10. Limitations. (1) The nudge and injection flags are regexes, so a new phrasing gets past them. (2) The agent sometimes searches the wrong corpus first, and it doesn't say when it can't see something like a user's specific PTO balance; instead it just replies with the generic policy's accrual table, and does not acknowledge this as a subpar answer that lacks personalization because it was missing information about the user. (3) When the user approves a command, nothing is logged when ideally always-allow approvals should be saved to memory.